AC-II

Aula 1

- microprocessadores vs microcontroladores
- sistemas embebidos
- desenvolvimento de aplicações para microcontroladores
- noção de periférico; estrutura básica de um modulo de I/O;
  registos de controlo; status e dados; modelo de programação

microprocessador vs microcontrolador

- circuito integrado com um ou mais CPU;
- não tem memória interna (além dos registos)
- os barramentos estão disponíveis no exterior
- é necessário adicionar RAM, ROM e periféricos para ficar completo
- opera em frequências altas (>3Ghz)
- Sistemas computacionais de uso geral

- circuito integrado que inclui CPU, RAM, ROM e periféricos
- funciona em frequências baixas
- baixo consumo de energia
- grande variedade de periféricos e interfaces com o exterior
- rapidez de resposta a eventos externos (sistemas de tempo real)
- utilizado em tarefas específicas
  Ex. controlo da velocidade de um motor)

Sistemas embebidos

- sistema computacional especializado
  realiza algo específico ou controla algo
- tem requisitos próprios e executa apenas tarefas pré-definidas
- recursos disponíveis mas mais limitados que num sistema computacional de uso geral (menos memória,...)
- custo inferior a um sistema computacional de uso geral

Ex. de utilização: eletrónica de consumo, automóveis
telecomunicações, domótica, robótica, iot

microcontrolador

dispositivo programável que integra num único circuito:

- Unidade de Processamento
- Memoria Volátil e Não Volátil
- Portos I/O (E/S)

e ainda inclui, timers, conversor A/D, Serial I/O.

barramentos (dados, endereços e controlo) interligam todos (não estão geralmente acessíveis externamente

versatilidade

externamente há, em geral, pinos que podem ser configurados

geração do código

em C ou assembly

é necessário utilizar cross-compilers / cross-assemblers

no desenvolvimento de aplicações, devido a recursos necessários

programa-monitor: é um programa que reside, de forma permanente, na memória não volátil

bootloader: é um programa que reside, de forma permanente, na memória do microcontrolador e que disponibiliza funções básicas de transferência e execução de um programa

in-circuit debugger (ICD): é um dispositivo hardware controlado por software do host que permite a transferência e execução controlada de um programa.

O ICD é normalmente, necessário para a transferência de um programa-monitor ou bootloader

tecnicas de memória não volátil

ROM - programada no processo de fabrico

PROM - programável uma única vez

EPROM - escrita em segundos, apagamento em minutos

EEPROM - o apagamento e a escrita podem ser efetuadas no mesmo circuito
o apagamento é feito byte a byte
escrita muito mais lenta que leitura

Flash EEPROM - semelhante a ↑
escrita pressupõe a inicialização prévia das zonas de memória a escrever (reset)
tecnologia mais rápida porque o reset pode ser feito em blocos.

O nosso microcontrolador

memória flash: 512 kB

memória RAM: 128 kB

Versão 64 pinos (também há 100 e 121)

CPU

32 registos de 32 bits

espaço de endereçamento de 32 bits

organização de memória: byte-addressable

max. freq. 80MHz

Microcontroladores baseados em Arquitetura de Harvard

→ dois espaços de endereçamento independentes: um para instruções, outro para dados

→ apenas o bloco ID tem acesso à memória

→ o programa não pode aceder a dados armazenados na memória de instruções

→ o CPU não pode executar instruções armazenadas na memória de dados

Solução PIC32 para limitações da Arquitetura de Harvard

Bus Matrix

é um comutador (switch) que funciona à mesma frequência do CPU

estabelece ligações ponto a ponto

entre o bloco IF do CPU e memória Flash

entre o bloco IF e a memória RAM

entre o bloco MEM do CPU e memória RAM

entre o bloco MEM do CPU e memória Flash

Assim, o espaço de endereçamento aparece como um espaço linear unificado (instruções e dados residem no mesmo espaço de endereçamento, cada um ocupando uma gama de endereços única)

Ou seja, para o programador é uma Arquitetura Von Neumann

dispositivos periféricos

estrutura básica de um módulo de I/O

registos de controlo, status e dados

modelo de programação

- assíncronos relativamente ao CPU
- geram diferentes quantidades de informação
  com diferentes formatos a diferentes velocidades
- mais lentos que o CPU e memória

é necessária uma interface que adapte características
do dispositivo periférico às do CPU/memória

modelo de programação do periférico

Data Register (Read/write)

Status Register (Read only)

Control Register (Write only ou Read/write)

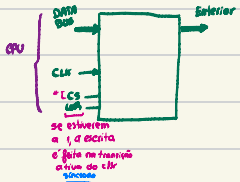
Comunicação entre CPU e outros dispositivos

protocolo
- Write (CPU → dispositivo externo)
- Read (CPU ← dispositivo externo)

aula 2

1 exemplo de periféricos simples
2 portos I/O no PIC32
3 transferência de informação entre periféricos e memória (polling)

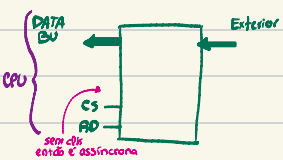
① porto de saída de 32 bits



- armazena informação vinda do CPU durante escrita na memória (sw)
  MEM
- o sinal CS é gerado pelo descodificador de endereços: fica ativo se o endereço gerado pelo CPU for igual ao endereço do porto

porto de entrada de 32 bits

- em geral portos de entrada não têm capacidade de armazenamento)



- a informação presente na entrada é transferida para o CPU durante uma operação de leitura
- as saídas têm portas tri-state, só são ativas se CS e RD estão ativos
  resposta

② o microcontrolar PIC32... disponibiliza vários portos I/O (de B a G) com várias dimensões, mas no máximo 16 bits

- cada um destes pode ser configurado como Input ou Output
- um porto I/O de n bits é um conjunto de n portos de I/O de 1 bit

cada um dos portos tem associado 12 registos de 32 bits, os que são usados em AC-II são:

- TRIS - configuração → bit n = 1, bit n configurado como entrada; bit n = 0, bit n configurado como saída
- PORT - leitura
- LAT - escrita

} 32 bits cada registo em que os 16 mais significativos são ignorados

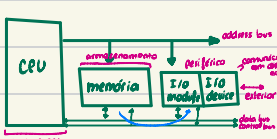
Detalhes

- cada porto inclui uma porta schmitt trigger (comparador com histerese) para melhorar a imunidade ao ruído
- no porto de entrada, o sinal externo é sincronizado através de dois flip-flops (S1 e S2) para resolver problemas de meta-estabilidade decorrentes pelo sinal externo ser assíncrono ao clk do CPU
- os dois flip-flops impõem um atraso de dois ciclos de relógio na propagação do sinal até ao barramento de dados do CPU

número de ciclos de relógio gastos pelo CPU em tarefas que não estão diretamente relacionadas com a transferência

o overhead deste método depende do número de vezes que é executado o polling

③



1. transferência de informação disco → memória
   1º efetuar o pedido
   2º esperar que o disco tenha a informação disponível
   3º transferir a informação da memória
2. Estes valores dependem da tecnologia do disco:
   Latência: tempo que decorre do pedido ao 1º byte de informação
   Taxa de transferência de pico (burst): nº máximo de bytes transferidos por segundo após latência
   Taxa de transferência: nº total de bytes transferidos / tempo total (incluindo latência)

3. técnicas de gestão de transferência de dados
   - E/S programada (programmed I/O)
     - CPU assume controlo total da transferência
     - CPU inicia operação, verifica repetidamente se o dispositivo está pronto para enviar/receber (polling) e efetua a transferência
     - polling: CPU fica em "espera ativa", pode desperdiçar ciclos de CPU

técnica básico que pode ser utilizada quando a velocidade do periférico não diminui drasticamente a capacidade de processamento do CPU

- E/S por interrupção (interrupt driven I/O)
  - o periférico sinaliza que está pronto
  - o CPU executa a transferência de dados enquanto aguarda a interrupção pode continuar a fazer outras tarefas
- E/S por acesso direto à memória (DMA)
  - um dispositivo externo ao CPU (DMA) assegura a transferência dos dados diretamente entre a memória e o periférico